

Python Charting

A brief overview of a **very** big topic

https://bioinfmsc8.bio.ed.ac.uk/AY25_BPSM19.html

Quick links:
Click on any 🏠 to get back here!

- all about `matplotlib`
- plot some points with different plotting characters and colours
- figure size (figure, figsize)
- add a legend , axis, title
- multiple lines on a picture
- specifying both axes: sinusoid curves
- plot characters
- multi character plots
- line properties
- do a multi-panel figure using subplot()

- histograms
- annotating the picture
- non-linear axes`
- sliding window plot`

Exercises

1. generate pictures from base composition data
2. make a user-interactive function

-
- important git/GitHub stuff at the end

🏠What do we need?

`matplotlib.pyplot` is a Python charting library that has its own extensive web site <http://matplotlib.org/>.

`seaborn` is a visualization library that is based on matplotlib; it's excellent for making pleasing-looking statistical graphics! It also has its own web site: <https://seaborn.pydata.org/>.

There are vast amounts of additional guidance available on line, e.g.

- <https://matplotlib.org/resources/index.html>
- <https://www.python-graph-gallery.com/matplotlib/>
- <https://www.geeksforgeeks.org/graph-plotting-in-python-set-1/>
- <https://seaborn.pydata.org/tutorial.html>
- https://s3.amazonaws.com/assets.datacamp.com/blog_assets/Python_Matplotlib_Cheat_Sheet.pdf

Let's get happy with matplotlib

To access the functionality of the `matplotlib` library, we must import it.

Today we will also want to use `numpy`, so we need to import that too.

`numpy` is an extension to the Python programming language, adding support for large, multi-dimensional arrays and matrices, along with a large library of high-level mathematical functions to operate on these arrays.

It also has its own web site too: www.numpy.org.

```
# A standard start for our session
```

```
import os, sys
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# What version do we have?
```

```
from importlib.metadata import version
```

```
version('matplotlib')
```

```
'3.6.3'
```

Notice how we're using `plt` as an `alias` for `matplotlib.pyplot`. This is so we can avoid typing `matplotlib.pyplot` every time we want to call a method.

We have done the same thing, i.e. create an alias, for `numpy`.

The name `plt` is a standard alias that you'll see many times in examples and documentation.

Getting help within Python

help(plt)

Help on module matplotlib.pyplot in matplotlib:

NAME

matplotlib.pyplot

DESCRIPTION

`matplotlib.pyplot` is a state-based interface to matplotlib. It provides an implicit, MATLAB-like, way of plotting. It also opens figures on your screen, and acts as the figure GUI manager.

pyplot is mainly intended for interactive plots and simple cases of programmatic plot generation::

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(0, 5, 0.1)
y = np.sin(x)
plt.plot(x, y)
```

The explicit object-oriented API is recommended for complex plots, though pyplot is still usually used to create the figure and often the axes in the figure. See `plt.figure`, `plt.subplots`, and `plt.subplot_mosaic` to create figures, and `plt.Axes` for the plotting methods on an `Axes`::

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(0, 5, 0.1)
y = np.sin(x)
fig, ax = plt.subplots()
ax.plot(x, y)
```

See `matplotlib.pyplot` for an explanation of the tradeoffs between the implicit and explicit interfaces.

FUNCTIONS

`acorr(x, *, data=None, **kwargs)`
Plot the autocorrelation of `x`.

Parameters

`x` : array-like

..... and lots more.

What modules do we have loaded?

dir()

```
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', 'np', 'os', 'plt', 'sys']
```

It's those double underscore ("dunders" or special variables/methods "magic") things again!? Here's some links to give

you more info:

- [underscores and things like that](#)
- [another link about using underscores](#)
- [and a rather larger topic, Python3 object classes](#)

... meanwhile ...

What does plt "have in it"?
dir(plt)

['Annotation', 'Arrow', 'Artist', 'AutoLocator', 'Axes', 'Button', 'Circle', 'Enum', 'ExitStack', 'Figure', 'FigureBase', 'FigureCanvasBase', 'FixedFormatter', 'FixedLocator', 'FormatStrFormatter', 'Formatter', 'FuncFormatter', 'GridSpec', 'IndexLocator', 'Line2D', 'LinearLocator', 'Locator', 'LogFormatter', 'LogFormatterExponent', 'LogFormatterMathtext', 'LogLocator', 'MaxNLocator', 'MouseButton', 'MultipleLocator', 'Normalize', 'NullFormatter', 'NullLocator', 'Number', 'PolarAxes', 'Polygon', 'Rectangle', 'ScalarFormatter', 'Slider', 'Subplot', 'SubplotSpec', 'Text', 'TickHelper', 'Widget', '_NON_PLOT_COMMANDS', '_REPL_DISPLAYHOOK', '_ReplDisplayHook', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__', '_api', '_auto_draw_if_interactive', '_backend_mod', '_copy_docstring_and_deprecators', '_docstring', '_draw_all_if_interactive', '_get_backend_mod', '_get_required_interactive_framework', '_interactive_bk', '_log', '_pylab_helpers', '_warn_if_gui_out_of_main_thread', 'acorr', 'angle_spectrum', 'annotate', 'arrow', 'autoscale', 'autumn', 'axes', 'axhline', 'axhspan', 'axis', 'axline', 'axvline', 'axvspan', 'bar', 'bar_label', 'barbs', 'barh', 'bone', 'box', 'boxplot', 'broken_barh', 'cbook', 'cla', 'clabel', 'clf', 'clim', 'close', 'cm', 'cohere', 'color_sequences', 'colorbar', 'colormaps', 'connect', 'contour', 'contourf', 'cool', 'copper', 'csd', 'cycler', 'delaxes', 'disconnect', 'draw', 'draw_all', 'draw_if_interactive', 'errorbar', 'eventplot', 'figaspect', 'figimage', 'figlegend', 'fignum_exists', 'figtext', 'figure', 'fill', 'fill_between', 'fill_betweenx', 'findobj', 'flag', 'functools', 'gca', 'gcf', 'gci', 'get', 'get_backend', 'get_cmap', 'get_current_fig_manager', 'get_figlabels', 'get_fignums', 'get_plot_commands', 'get_scale_names', 'getp', 'ginput', 'gray', 'grid', 'hexbin', 'hist', 'hist2d', 'hlines', 'hot', 'hsv', 'importlib', 'imread', 'imsave', 'imshow', 'inferno', 'inspect', 'install_repl_displayhook', 'interactive', 'ioff', 'ion', 'isinteractive', 'jet', 'legend', 'locator_params', 'logging', 'loglog', 'magma', 'magnitude_spectrum', 'margins', 'matplotlib', 'matshow', 'minorticks_off', 'minorticks_on', 'mlab', 'new_figure_manager', 'nipy_spectral', 'np', 'pause', 'pcolor', 'pcolormesh', 'phase_spectrum', 'pie', 'pink', 'plasma', 'plot', 'plot_date', 'polar', 'prism', 'psd', 'quiver', 'quiverkey', 'rc', 'rcParams', 'rcParamsDefault', 'rcParamsOrig', 'rc_context', 'rcdefaults', 'rcsetup', 're', 'register_cmap', 'rgrids', 'savefig', 'sca', 'scatter', 'sci', 'semilogx', 'semilogy', 'set_cmap', 'set_loglevel', 'setp', 'show', 'specgram', 'spring', 'spy', 'stackplot', 'stairs', 'stem', 'step', 'streamplot', 'style', 'subplot', 'subplot2grid', 'subplot_mosaic', 'subplot_tool', 'subplots', 'subplots_adjust', 'summer', 'suptitle', 'switch_backend', 'sys', 'table', 'text', 'thetagrids', 'threading', 'tick_params', 'ticklabel_format', 'tight_layout', 'time', 'title', 'tricontour', 'tricontourf', 'tripcolor', 'tripplot', 'twinx', 'twiny', 'uninstall_repl_displayhook', 'violinplot', 'viridis', 'vlines', 'waitforbuttonpress', 'winter', 'xcorr', 'xkcd', 'xlabel', 'xlim', 'xscale', 'xticks', 'ylabel', 'ylim', 'yscale', 'yticks']

Or we could do the DOT TAB TAB thing

plt.			
Display all 233 possibilities? (y or n)			
plt.Annotation()	plt.barh()	plt.hist2d()	plt.semilogx()
plt.Arrow()	plt.bone()	plt.hlines()	plt.semilogy()
plt.Artist()	plt.box()	plt.hot()	plt.set_cmap()
plt.AutoLocator()	plt.boxplot()	plt.hsv()	plt.set_loglevel()
plt.Axes()	plt.broken_barh()	plt.importlib	plt.setp()
plt.Button()	plt.cbook	plt.imread()	plt.show()
plt.Circle()	plt.cla()	plt.imsave()	plt.specgram()
plt.Enum()	plt.clabel()	plt.imshow()	plt.spring()
plt.ExitStack()	plt.clf()	plt.inferno()	plt.spy()
plt.Figure()	plt.clim()	plt.inspect	plt.stackplot()
plt.FigureBase()	plt.close()	plt.install_repl_displayhook()	plt.stairs()
plt.FigureCanvasBase()	plt.cm	plt.interactive()	plt.stem()
plt.FixedFormatter()	plt.cohere()	plt.ioff()	plt.step()
plt.FixedLocator()	plt.color_sequences	plt.ion()	plt.streamplot()
plt.FormatStrFormatter()	plt.colorbar()	plt.isinteractive()	plt.style
plt.Formatter()	plt.colormaps()	plt.jet()	plt.subplot()
plt.FuncFormatter()	plt.connect()	plt.legend()	plt.subplot2grid()
plt.GridSpec()	plt.contour()	plt.locator_params()	plt.subplot_mosaic()
plt.IndexLocator()	plt.contourf()	plt.logging	plt.subplot_tool()
plt.Line2D()	plt.cool()	plt.loglog()	plt.subplots()
plt.LinearLocator()	plt.copper()	plt.magma()	plt.subplots_adjust()
plt.Locator()	plt.csd()	plt.magnitude_spectrum()	plt.summer()
plt.LogFormatter()	plt.cycler()	plt.margins()	plt.suptitle()
plt.LogFormatterExponent()	plt.delaxes()	plt.matplotlib	plt.switch_backend()
plt.LogFormatterMathtext()	plt.disconnect()	plt.matshow()	plt.sys

There are three methods we need to know about.

- ```
Let's be tidy!
os.getcwd()
'/localdisk/home/aivens2'
os.mkdir("Exercises/Lecture19")
os.chdir("Exercises/Lecture19")
os.getcwd()
'/localdisk/home/aivens2/Exercises/Lecture19'

Below is a list of numbers for the Y axis
It is assumed that the X axis is 1, 2, 3, etc.
Create the plot object
plt.plot([1,5,6,8,4,6,7,30,1,2,6,4,9,8,1])

Nothing has been drawn!!

This is all we get told, and we get something
"similar" to this each time we use plt
[<matplotlib.lines.Line2D object at 0x7fd09a11c278>]

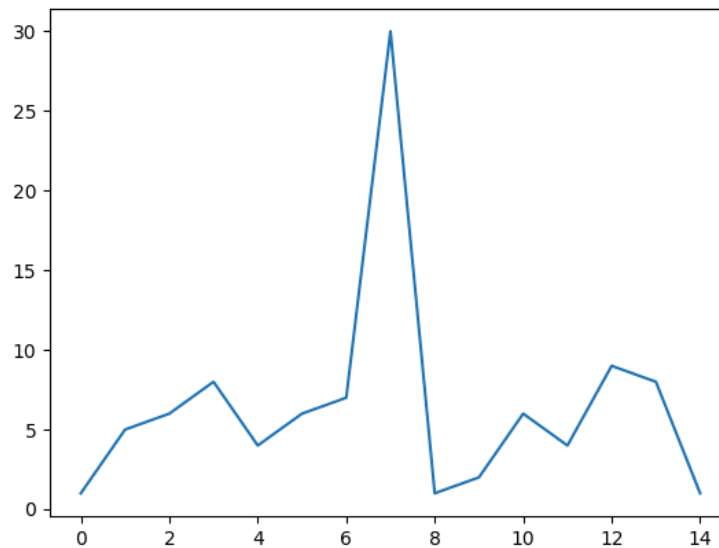
Save the figure, format given by the filename suffix"
```

```
plt.savefig("Chart_01.png",transparent=True)
```

```
Show the image
```

```
plt.show()
```

```
On your machines it will/might pop up in a separate window,
that you have to close to get the python command line back!
```



```
What happens if you try to show the plot again?
```

```
plt.show()
```

**SAVE BEFORE SHOW!**

```
That image pop-up "problem"....
```

```
plt.plot([1,5,6,8,4,6,7,30,1,2,6,4,9,8,1])
```

```
plt.show(block=False)
```

```
Image is ok, close it
```

```
plt.close()
```

The plot() method can take a lot of different optional arguments.

Loads of [documentation](#) here....

```
We can change the colour of the line (note that it is the American spelling of color!)
```

```
A list of numbers for the Y axis
```

```
numbers = [1,5,6,8,4,6,7,3,1,2,6,4,9,8,1]
```

```
Generate
```

```
plt.plot(numbers, color="green")
```

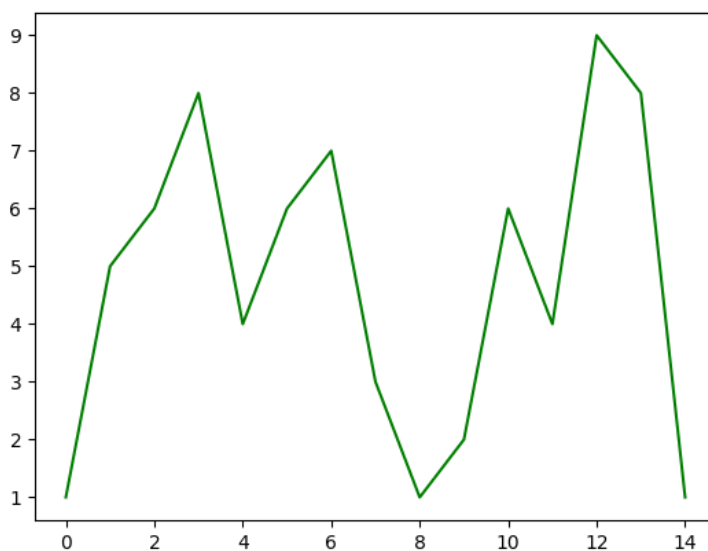
```
[<matplotlib.lines.Line2D object at 0x7fee908da6a0>]
```

```
Save to file
```

```
plt.savefig("Chart_02.png",transparent=True)
```

```
Show the image
```

```
plt.show()
```



```
We need a legend, and to do that, we need a label keyword argument
```

```
so that we can attach a label to the points
```

```
A list of numbers for the Y axis
```

```
numbers = [1,5,6,8,4,6,7,3,1,2,6,4,9,8,1]
```

```
Generate, with label argument
```

```
plt.plot(numbers, color="red", label="My numbers")
```

```
[<matplotlib.lines.Line2D object at 0x7fee908cd710>]
```

```
Add legend
```

```
plt.legend(loc='upper right')
```

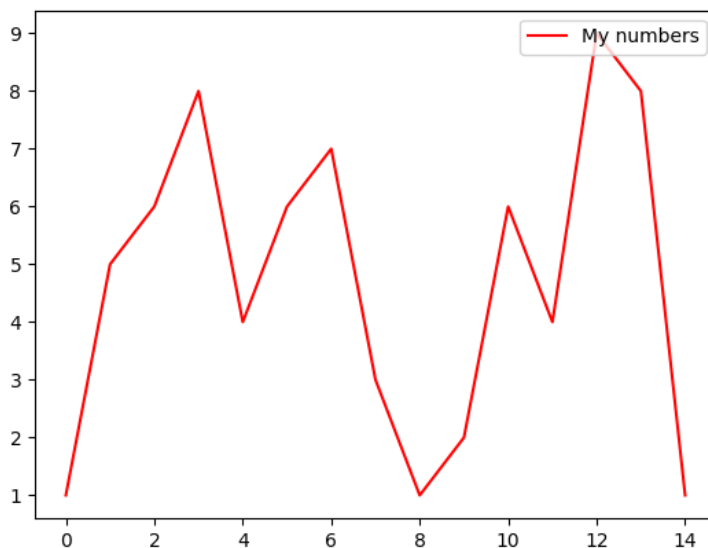
```
<matplotlib.legend.Legend object at 0x7fee908da048>
```

```
Save to file
```

```
plt.savefig("Chart_03.png",transparent=True)
```

```
Show the image
```

```
plt.show()
```



```
We can make the chart bigger by calling the figure()
```

```
method with the figsize argument
```

```
We HAVE to do this first before adding plot elements.
```

```
We'll also add some axis labels and a title
```

```
A list of numbers for the Y axis
```

```
numbers = [1,5,6,8,4,6,7,3,1,2,6,4,9,8,1]
```

```
Set the figure size (defaults to inches, at 100dpi resolution)
```

```
plt.figure(figsize=(20,10))
```

```
<Figure size 2000x1000 with 0 Axes>
```

**# Generate, with label argument**

```
plt.plot(numbers, color="red", label="My numbers")
```

```
[<matplotlib.lines.Line2D object at 0x7fee90e83588>]
```



**# Add legend, axis labels, title**

```
plt.legend(loc='upper right')
```

```
<matplotlib.legend.Legend object at 0x7fee91205a20>
```

```
plt.xlabel('X axis values')
```

```
Text(0.5, 0, 'X axis values')
```

```
plt.ylabel('Y axis values')
```

```
Text(0, 0.5, 'Y axis values')
```

```
plt.title('A matplotlib chart')
```

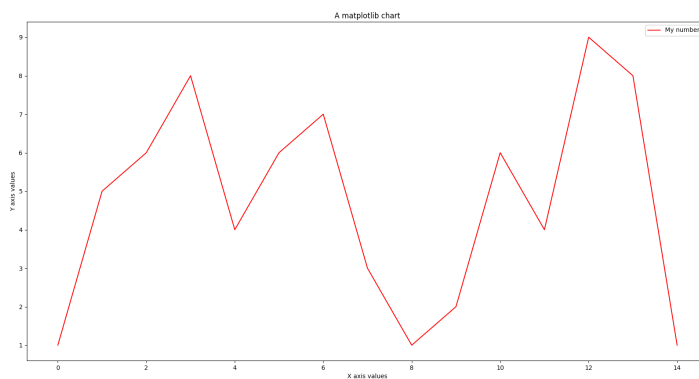
```
Text(0.5, 1.0, 'A matplotlib chart')
```

**# Save to file**

```
plt.savefig("Chart_04.png",transparent=True)
```

**# Show the image**

```
plt.show()
```



**# We can draw multiple sets of data on the same image**

**# matplotlib will automatically give each one a different colour**

**# Set the figure size**

```
plt.figure(figsize=(10,5))
```

```
<Figure size 1000x500 with 0 Axes>
```

**# First list of numbers**

```
numbers = [1,5,6,8,4,6,7,3,1,2,6,4,9,8,1]
```

```
Generate, with label argument
```

```
plt.plot(numbers, label="My numbers")
```

```
[<matplotlib.lines.Line2D object at 0x7fee9069fd68>]
```

```
Second list of numbers
```

```
more_numbers = [8,9,1,2,6,4,8,6,5,8,4,3,5,4,3]
```

```
Generate, with label argument
```

```
plt.plot(more_numbers, label="My other numbers")
```

```
[<matplotlib.lines.Line2D object at 0x7fee908c0470>]
```

```
Third list of numbers, longer than the first two
```

```
even_more_numbers = list(range(0,25))
```

```
even_more_numbers
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24]
```

```
Generate, with label argument
```

```
plt.plot(even_more_numbers, label="My range of numbers")
```

```
[<matplotlib.lines.Line2D object at 0x7fee9069f128>]
```

```
Add legend, axis labels
```

```
plt.legend()
```

```
<matplotlib.legend.Legend object at 0x7fee908e06d8>
```

```
plt.xlabel('X axis values')
```

```
Text(0.5, 0, 'X axis values')
```

```
plt.ylabel('Y axis values')
```

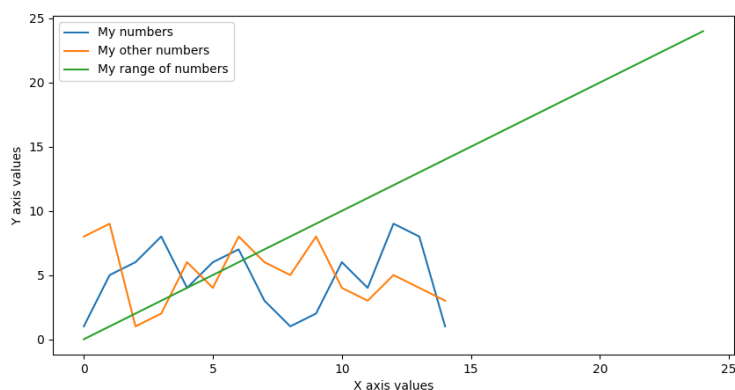
```
Text(0, 0.5, 'Y axis values')
```

```
Save to file
```

```
plt.savefig("Chart_05.png",transparent=True)
```

```
Show the image
```

```
plt.show()
```



## ⬆Specifying the data for both axes: a sinusoid curve

```
We can use numpy to generate 1000 evenly-spaced values from 0 to 20
```

```
x = np.linspace(0, 20, 1000)
```

```
We can use numpy to generate sine values for the x values we have made
```

```
y = np.sin(x)
```

```
What is the x input?
```

```
x
```

```
array([0. , 0.02002002, 0.04004004, 0.06006006,
 0.08008008, 0.1001001 , 0.12012012, 0.14014014,
 0.16016016, 0.18018018, 0.2002002 , 0.22022022,
 0.24024024, 0.26026026, 0.28028028, 0.3003003 ,
 0.32032032, 0.34034034, 0.36036036, 0.38038038,
 0.4004004 , 0.42042042, 0.44044044, 0.46046046,
 ])
```

```
type(x)
```

```
<class 'numpy.ndarray'>
```

```
What is the y input?
```

```
y
```

```
array([0. , 0.02001868, 0.04002934, 0.06002396, 0.07999452,
 0.09993302, 0.11983146, 0.13968188, 0.15947632, 0.17920684,
 0.19886554, 0.21844453, 0.23793597, 0.25733206, 0.276625 ,
 0.29580708, 0.3148706 , 0.33380793, 0.35261147, 0.37127369,
 0.3897871 , 0.4081443 , 0.42633791, 0.44436066, 0.46220531,
 0.47986471, 0.49733179, 0.51459954, 0.53166105, 0.54850948,
 ])
```

```
type(y)
```

```
<class 'numpy.ndarray'>
```

```
Generate the plot
```

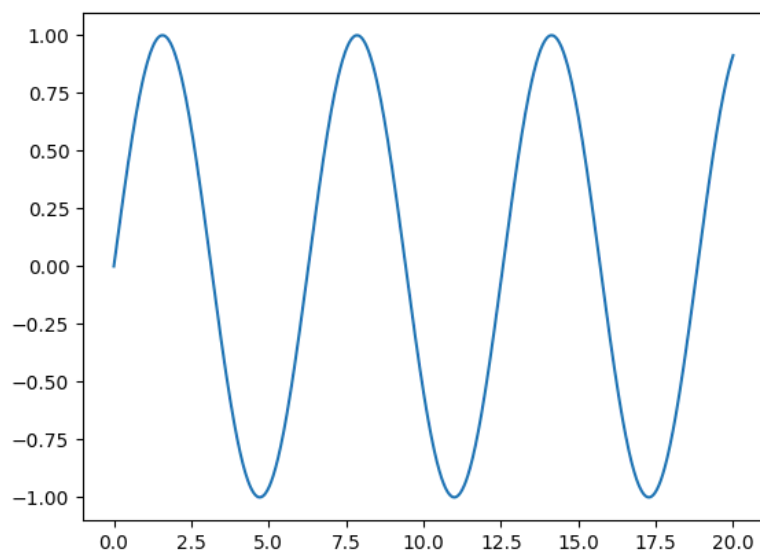
```
plt.plot(x, y)
```

```
[<matplotlib.lines.Line2D object at 0x7fee90e7`6f98>]
```

```
Save and show
```

```
plt.savefig("Chart_14.png",transparent=True)
```

```
plt.show()
```



### 🏠 How do we want the points to look?

For every x, y pair of arguments, there is an optional third argument which is the format string that indicates the color and line type of the plot.

The letters and symbols of the format string are from MATLAB, and you concatenate a color string with a line style string.

The default format string is 'b-', which is a solid blue line.

Some options for the color characters are:

```
'r' = red
'g' = green
'b' = blue
'c' = cyan
'm' = magenta
'y' = yellow
'k' = black
'w' = white
```

Some options for line styles are

```
'-' = solid
 '--' = dashed
 ':' = dotted
 '-.' = dot-dashed
 '.' = points
 'o' = filled circles
 '^' = filled triangles
```

```
We want the points as red circles
```

```
plt.plot([1,2,3,4], [1,4,9,16], 'ro')
```

```
[<matplotlib.lines.Line2D object at 0x7fee90e76860>]
```

```
The axis() command takes a list of [xmin, xmax, ymin, ymax] and specifies the viewport of the axes.
```

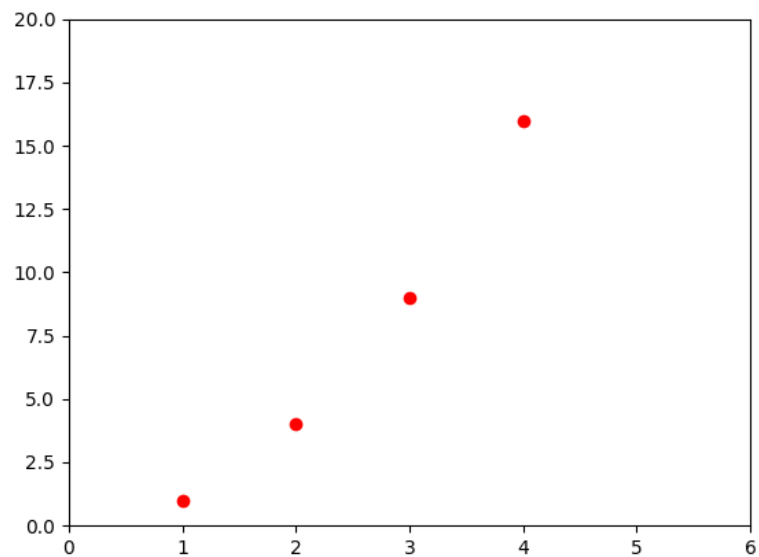
```
plt.axis([0, 6, 0, 20])
```

```
[0, 6, 0, 20]
```

```
Save and show
```

```
plt.savefig("Chart_06.png",transparent=True)
```

```
plt.show()
```



## 🏠 Multi-character plots

```
Say we want evenly sampled timepoints at 200ms intervals, so lets use a range
```

```
list(range(0., 5., 0.2))
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
TypeError: 'float' object cannot be interpreted as an integer
```

```
Using numpy to generate a range
```

```
t = np.arange(0., 5., 0.2)
```

```
t
```

```
array([0. , 0.2, 0.4, 0.6, 0.8, 1. , 1.2, 1.4, 1.6, 1.8, 2. ,
 2.2, 2.4, 2.6, 2.8, 3. , 3.2, 3.4, 3.6, 3.8, 4. , 4.2,
 4.4, 4.6, 4.8])
```

```
Generate three lines, with different plotting characters
```

```
We want red dashes, blue squares and green triangles!
```

```
plt.plot(t, t, 'r--', t, t**2, 'bs', t, t**3, 'g^')
```

```
[<matplotlib.lines.Line2D object at 0x7fee9069f7b8>, <matplotlib.lines.Line2D object at 0x7fee9069fe10>,
<matplotlib.lines.Line2D object at 0x7fee9069f2e8>]
```

```
Add axis labels
```

```
plt.xlabel('X axis values')
```

```
Text(0.5, 0, 'X axis values')
```

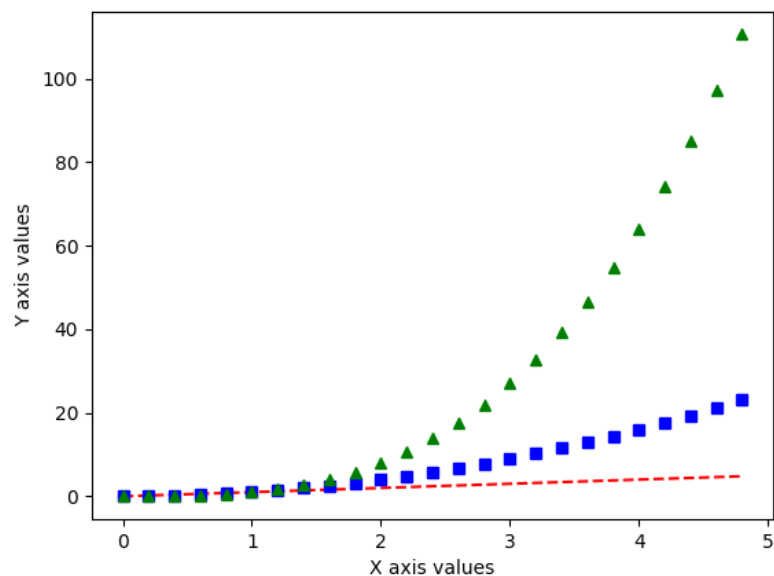
```
plt.ylabel('Y axis values')
```

```
Text(0, 0.5, 'Y axis values')
```

```
Save and show
```

```
plt.savefig("Chart_07.png", transparent=True)
```

```
plt.show()
```



## 🏠Line properties

Lines have many attributes that you can set: linewidth, dash style, antialiased, etc. For a description of the many, many details, this is a good reference [matplotlib.lines.Line2D](#). There are several ways to set line properties

- Use keyword args

```
plt.plot(x, y, linewidth=2.0)
```

- Use the `setp()` command (set properties)

```
plot() returns a list of Line2D objects, where x, y etc are arrays
```

```
a = x+2
```

```
b = y*5
```

```
setp() works transparently with a list of objects or a single object
```

```
line1, line2 = plt.plot(x, y, a, b)
```

```
plt.setp(line1, color='r', linewidth=6.0)
```

```
[None, None]
```

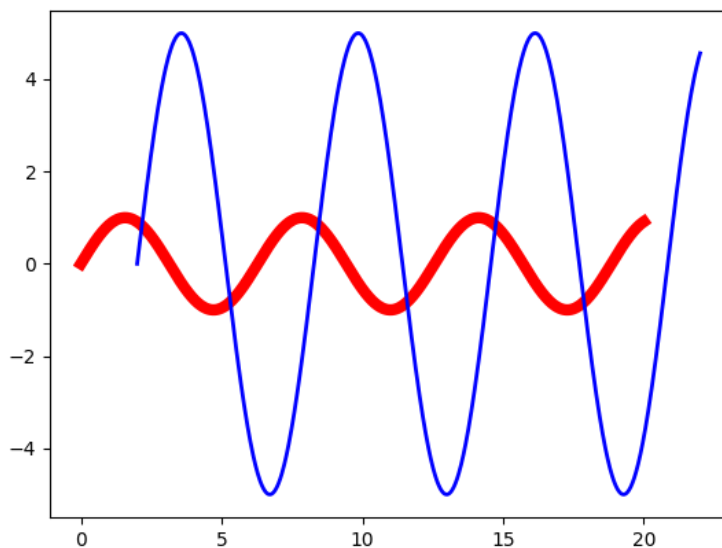
```
plt.setp(line2, color='b', linewidth=2.0)
```

```
[None, None]
```

```
Save and show
```

```
plt.savefig("Chart_77.png",transparent=True)
```

```
plt.show()
```



## 🏠 Working with multiple figures and axes

Plot objects can hold multiple plots, and we use `figure()` to indicate which figure it is.

`figure(1)` is created by default.

Panels or subplots are generated using the `subplot()` command.

`subplot()` specifies

- `numrows`,
- `numcols`,
- `fignum`

where *fignum* ranges from 1 to *numrows* \* *numcols*.

The commas in the subplot command are optional if *numrows* \* *numcols* is less than 10.

`subplot(211)` is identical to `subplot(2, 1, 1)` and means 2 rows, 1 column, first panel.

---

```
Here's a function
```

```
def f(t):
 return np.exp(-t) * np.cos(2* np.pi *t)
```

```
np.pi
```

```
3.141592653589793
```

```
Two numpy ranges
```

```
t1 = np.arange(0.0, 5.0, 0.1)
```

```
t2 = np.arange(0.0, 5.0, 0.02)
```

```
First figure
```

```
plt.figure(1)
```

```
First panel
```

```
plt.subplot(211)
```

```
plt.plot(t1, f(t1), 'bo', t2, f(t2), 'k')
```

```
Second panel
```

```
plt.subplot(212)
```

```
plt.plot(t2, np.cos(2*np.pi*t2), 'r--')
```

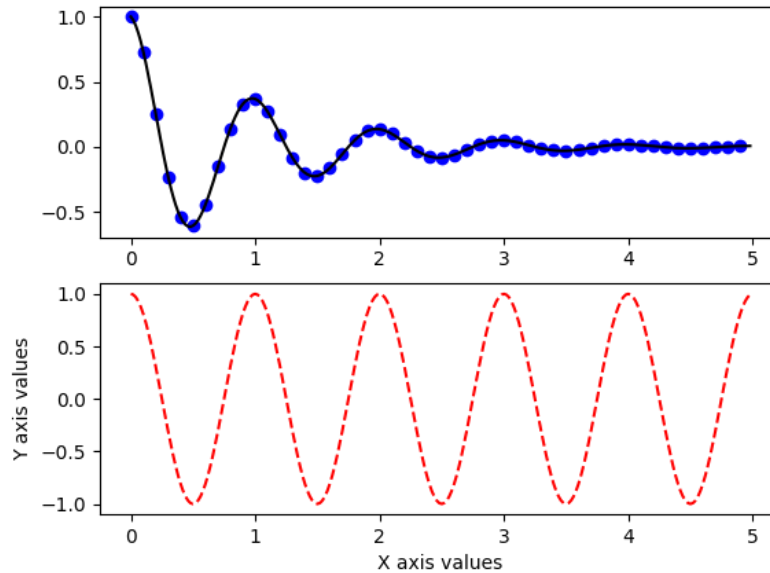
```
These apply to the panel we are working on (212)
```

```
plt.xlabel('X axis values')
```

```
plt.ylabel('Y axis values')
```

```
plt.savefig("Chart_08.png",transparent=True)
```

```
plt.show()
```



There are a lot of things that you can do with axes using the `axes()` command, which allows you to specify the location as `axes([left, bottom, width, height])` where all values are in fractional (0 to 1) coordinates. There are good examples available for

- [placing axes manually](#)
- [doing lots of subplots](#)

You can create multiple figures by using multiple `figure()` calls with an increasing figure number. Of course, each figure can contain as many axes and subplots as your heart desires!

```
The first figure, default size
```

```
plt.figure(1)
```

```
<Figure size 640x480 with 0 Axes>
```

```
Make the first subplot in the first figure
```

```
plt.subplot(211)
```

```
<matplotlib.axes._subplots.AxesSubplot object at 0x7f078ae3def0>
```

```
plt.plot([1, 2, 3])
```

```
[]
```

```
Make the second subplot in the first figure
```

```
plt.subplot(212)
```

```
<matplotlib.axes._subplots.AxesSubplot object at 0x7f0788681fd0>
```

```
plt.plot([4, 5, 6])
```

```
[]
```

```
The second figure, also default size
```

```
plt.figure(2)
```

```
<Figure size 640x480 with 0 Axes>
```

```
Create a subplot(111) by default
```

```
plt.plot([7, 8, 9])
```

```
[]
```

```
Make figure 1 current; subplot(212) still current
```

```
plt.figure(1)
```

```
<Figure size 640x480 with 2 Axes>
```

```
Make subplot(211) in figure1 current
```

```
plt.subplot(211)
```

```
<matplotlib.axes._subplots.AxesSubplot object at 0x7f078ae3def0>
```

```
Give subplot 211 a title
```

```
plt.title('Easy as 1, 2, 3')
```

```
Text(0.5, 1.0, 'Easy as 1, 2, 3')
```

```
Need to know which is which, so let's have them as variables
```

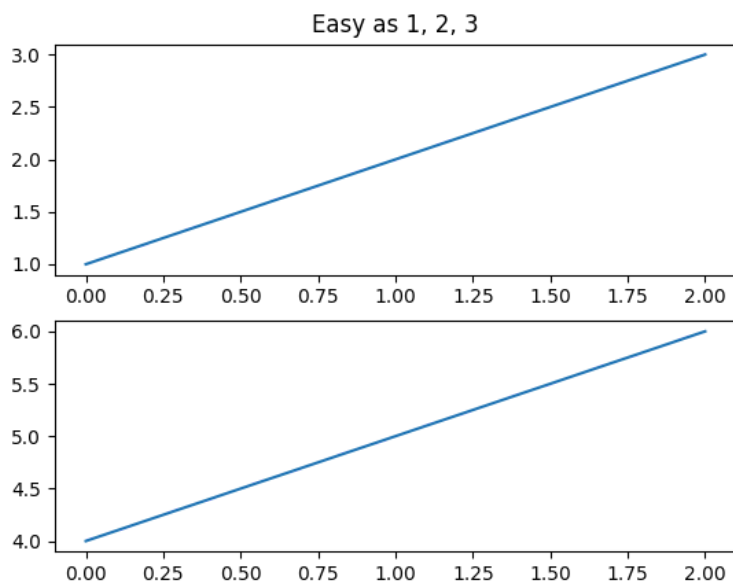
```
fig1 = plt.figure(1)
```

```
fig2 = plt.figure(2)
```

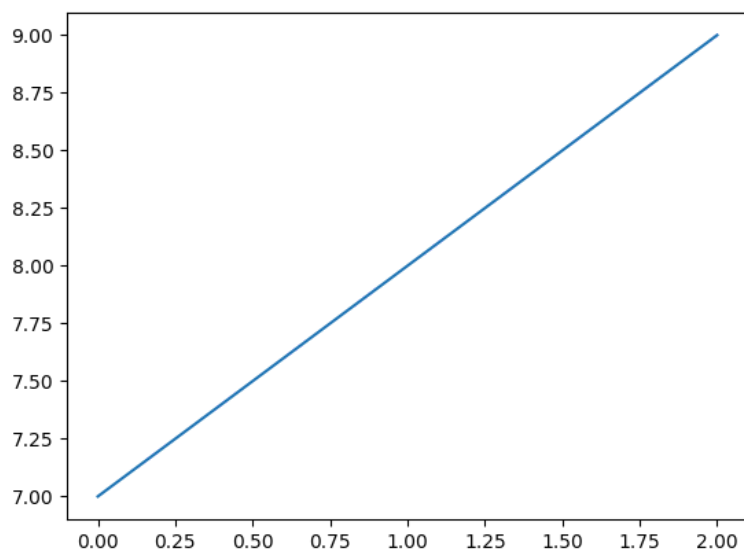
```
fig1.savefig("Chart_09.png",transparent=True)
```

```
fig2.savefig("Chart_10.png",transparent=True)
```

**figure(1) plot**



**figure(2) plot**



---

### 🏠Working with text()

There is an excellent summary of options given at:

[http://matplotlib.org/api/pyplot\\_summary.html](http://matplotlib.org/api/pyplot_summary.html)

The `text()` command can be used to add text in an arbitrary location, and the `xlabel()`, `ylabel()` and `title()` are used to add text in the indicated locations

#### # A simple numpy-generated dataset of IQ values for plotting

**mu, sigma = 100, 15**

**x = mu + sigma \* np.random.randn(10000)**

**x**

```
array([98.24955977, 99.96911731, 127.24351963, ..., 95.33318141,
 81.20784751, 79.73725928])
```

#### # Using plt.hist ....LOTS of options....

##### help(plt.hist)

`hist(x, bins=None, range=None, density=False, weights=None, cumulative=False, bottom=None, histtype='bar', align='mid', orientation='vertical')`  
Compute and plot a histogram.

This method uses `numpy.histogram` to bin the data in `*x*` and count the number of values in each bin, then draws the distribution either as a `BarContainer` or `Polygon`. The `*bins*`, `*range*`, `*density*`, and `*weights*` parameters are forwarded to `numpy.histogram`.

If the data has already been binned and counted, use `~.bar` or `~.stairs` to plot the distribution::

```
counts, bins = np.histogram(x)
plt.stairs(counts, bins)
```

Alternatively, plot pre-computed bins and counts using `hist()` by treating each bin as a single point with a weight equal to its count::

```
plt.hist(bins[:-1], bins, weights=counts)
```

The data input `*x*` can be a singular array, a list of datasets of potentially different lengths (`[*x0*, *x1*, ...]`), or a 2D ndarray in which each column is a dataset. Note that the ndarray form is transposed relative to the list form. If the input is an array, then the return value is a tuple (`*n*`, `*bins*`, `*patches*`); if the input is a sequence of arrays, then the return value is a tuple (`[*n0*, *n1*, ...]`, `*bins*`, `[*patches0*, *patches1*, ...]`).

Masked arrays are not supported.

##### Parameters

`x` : (n,) array or sequence of (n,) arrays  
Input values, this takes either a single array or a sequence of arrays which are not required to be of the same length.

`bins` : int or sequence or str, default: `rc:'hist.bins'`  
If `*bins*` is an integer, it defines the number of equal-width bins in the range.

If `*bins*` is a sequence, it defines the bin edges, including the left edge of the first bin and the right edge of the last bin; in this case, bins may be unequally spaced. All but the last

.... LOTS more....

#### # Make a histogram of the data, produces three sets of values

**n, bins, patches = plt.hist(x, 50, density=True, facecolor='#ee42f4', alpha=0.75)**

#### # What are they?

- `n`: is the number of counts (normally!) in each bin of the histogram, this time it is density
- `bins`: is the left hand edge of each bin
- `patches`: the individual patches used to create the histogram, e.g a collection of rectangles
- `facecolor`: what colour do we want? can use [hex values](#)

**n**

```
array([4.19784775e-05, 4.19784775e-05, 0.00000000e+00, 0.00000000e+00,
 8.39569551e-05, 2.93849343e-04, 2.93849343e-04, 2.09892388e-04,
 3.35827820e-04, 1.00748346e-03, 1.13341889e-03, 1.59518215e-03,
 2.89651495e-03, 3.27432125e-03, 3.86201993e-03, 6.75853488e-03,
 7.47216900e-03, 1.04106624e-02, 1.21737585e-02, 1.39368545e-02,
 1.54480797e-02, 2.10312172e-02, 2.06534109e-02, 2.30042057e-02,
 2.39277322e-02, 2.58587422e-02, 2.72440319e-02, 2.58587422e-02,
 2.69501826e-02, 2.36758613e-02, 2.27943133e-02, 1.82186593e-02,
 1.69173264e-02, 1.44405963e-02, 1.18799091e-02, 8.68954485e-03,
 7.51414748e-03, 6.29677163e-03, 3.94597689e-03, 3.69410602e-03,
 1.97298844e-03, 1.38528976e-03, 9.23526506e-04, 5.87698685e-04,
 4.19784775e-04, 2.09892388e-04, 1.25935433e-04, 8.39569551e-05,
 8.39569551e-05, 1.25935433e-04])
```

**bins**

```
array([37.03996302, 39.42213612, 41.80430922, 44.18648232,
 46.56865542, 48.95082853, 51.33300163, 53.71517473,
 56.09734783, 58.47952093, 60.86169403, 63.24386713,
 65.62604023, 68.00821333, 70.39038643, 72.77255953,
 75.15473264, 77.53690574, 79.91907884, 82.30125194,
 84.68342504, 87.06559814, 89.44777124, 91.82994434,
 94.21211744, 96.59429054, 98.97646364, 101.35863674,
 103.74080985, 106.12298295, 108.50515605, 110.88732915,
 113.26950225, 115.65167535, 118.03384845, 120.41602155,
 122.79819465, 125.18036775, 127.56254085, 129.94471396,
 132.32688706, 134.70906016, 137.09123326, 139.47340636,
 141.85557946, 144.23775256, 146.61992566, 149.00209876,
 151.38427186, 153.76644496, 156.14861806])
```

**patches**

<BarContainer object of 50 artists>

I'm not showing all the Python responses now...

**# Axis labelling etc**

```
plt.xlabel('Intelligence quotient value')
```

```
plt.ylabel('Probability')
```

```
plt.title('Histogram of IQ')
```

**# Axis ranges, tick marks (x1,x2 y1,y2)**

```
plt.axis([40, 160, 0, 0.03])
```

**# Annotation, note use of special np characters \mu and \sigma**

```
plt.text(60, .025, r'$\mu=100,\ \sigma=15$')
```

**# Additional region added on chart (vertical line)**

```
plt.axvline(x=mu+(2*sigma),linewidth=4, color='r')
```

**# Annotation for region added on chart**

```
plt.text(mu+(2.1*sigma), .025, r'$\mu + 2\sigma$')
```

**# Put a default grid on the chart**

```
plt.grid(True)
```

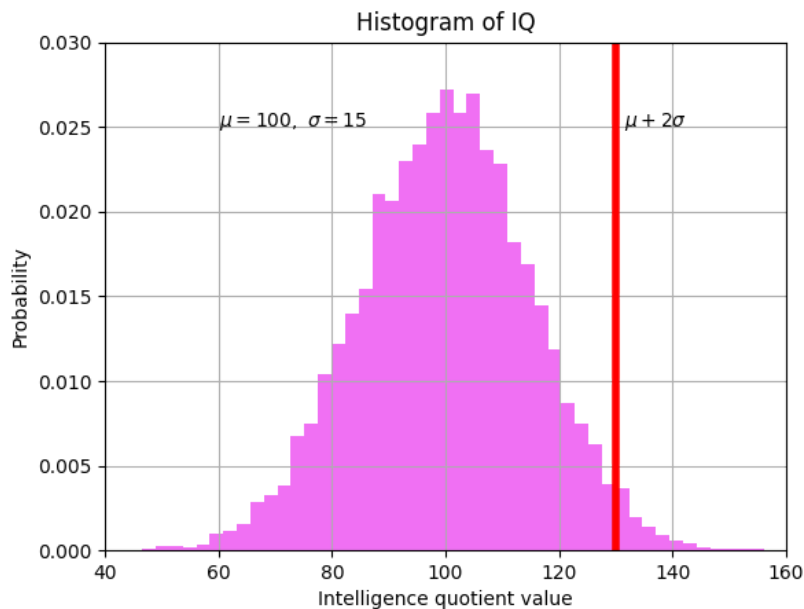
```
Could have changed things, e.g colour of the 9th rectangle...
```

```
plt.setp(patches[10], facecolor='g')
```

```
Save and show
```

```
plt.savefig("Chart_11.png",transparent=True)
```

```
plt.show()
```



## 📍Annotating text

Sometimes we want to draw attention to some feature of the plot, and the `annotate()` method provides helper functionality to make these annotations easy.

In an annotation, there are main two points to consider:

1. the location being annotated represented by the argument `xy`
2. the location of the text `xytext`

Don't forget there is lots of in-built help if you can't quite remember things....

```
help(plt.annotate)
```

Help on function annotate in module matplotlib.pyplot:

Annotate the point `*xy*` with text `*text*`.

In the simplest form, the text is placed at `*xy*`.

Optionally, the text can be displayed in another position `*xytext*`.  
An arrow pointing from the text to the annotated point `*xy*` can then be added by defining `*arrowprops*`.

```
Parameters

text : str
 The text of the annotation.

xy : (float, float)
 The point *(x, y)* to annotate. The coordinate system is determined
 by *xycoords*.

xytext : (float, float), default: *xy*
 The position *(x, y)* to place the text at. The coordinate system
 is determined by *textcoords*.

xycoords : single or two-tuple of str or `.Artist` or `.Transform` or callable, default: 'data'

 The coordinate system that *xy* is given in. The following types
 of values are supported:

 - One of the following strings:

=====
Value Description
=====
'figure points' Points from the lower left of the figure
'figure pixels' Pixels from the lower left of the figure
'figure fraction' Fraction of figure from lower left
'subfigure points' Points from the lower left of the subfigure
'subfigure pixels' Pixels from the lower left of the subfigure
'subfigure fraction' Fraction of subfigure from lower left
'axes points' Points from lower left corner of axes
'axes pixels' Pixels from lower left corner of axes
'axes fraction' Fraction of axes from lower left
'data' Use the coordinate system of the object
 being annotated (default)

.... etc
```

#### # Another numpy-generated dataset for plotting

```
t = np.arange(0.0, 5.0, 0.01)
```

```
s = np.cos(2* np.pi *t)
```

```
line = plt.plot(t, s, lw=2)
```

#### # Add some (not at all serious) text to the figure

```
for peak in range(1,5):
 plt.annotate('Pointy bits', xy=(peak, 1), xytext=(2.5, 1.5),
 arrowprops=dict(facecolor='#42f442', shrink=0.05),
)
```

```
Text(2.5, 1.5, 'Pointy bits')
```

```
Text(2.5, 1.5, 'Pointy bits')
```

```
Text(2.5, 1.5, 'Pointy bits')
```

```
Text(2.5, 1.5, 'Pointy bits')
```

#### # Set the y axis limits

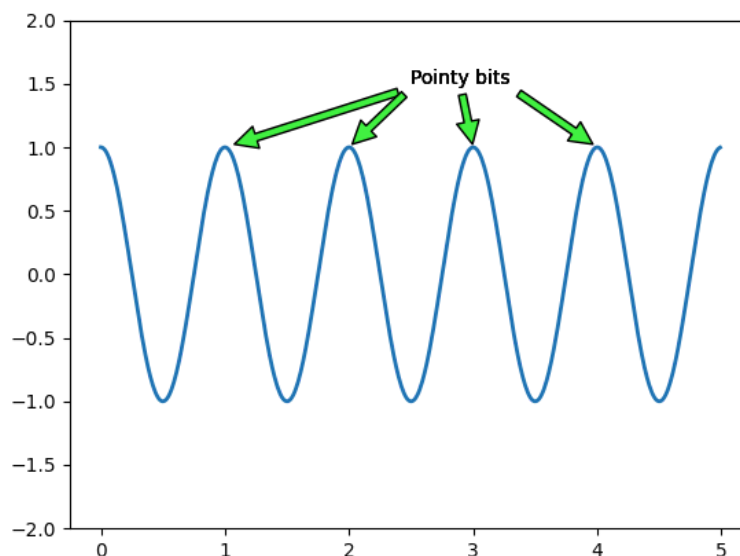
```
plt.ylim(-2,2)
```

```
(-2, 2)
```

#### # Save and show

```
plt.savefig("Chart_12.png",transparent=True)
```

```
plt.show()
```



## 📈 Logarithmic and other nonlinear axes

`plt` supports not only linear axis scales, but also logarithmic and logit ( $\log(z / (1 - z))$  for any value between 0 and 1) scales. This is commonly used if data span many orders of magnitude.

```
An example of four plots of the same data but different scales for the y axis
```

```
Make up some data in the interval 0 .. 1
```

```
y = np.random.normal(loc=0.5, scale=0.4, size=1000)
```

```
len(y)
```

```
1000
```

```
y = y[(y > 0) & (y < 1)]
```

```
len(y)
```

```
780
```

```
y.sort()
```

```
x = np.arange(len(y))
```

```
Plot with various axes scales
```

```
plt.figure(figsize=(20,10))
```

```
Linear
```

```
plt.subplot(221)
```

```
plt.plot(x, y, 'r-',linewidth=3.0)
```

```
plt.yscale('linear')
```

```
plt.title('LINEAR')
```

```
plt.grid(True)
```

```
Log
```

```
plt.subplot(222)
```

```
plt.plot(x, y, 'g-',linewidth=3.0)
```

```
plt.yscale('log')
```

```
plt.title('LOG')
```

```
plt.grid(True)
```

```
Symmetric log
```

```
plt.subplot(223)
```

```
plt.plot(x, y - y.mean(), 'm-',linewidth=3.0)
```

```
plt.yscale('symlog', linthresh=0.05)
```

```
plt.title('SYMLOG')
```

```
plt.grid(True)
```

```
Logit: log-odds, or the logarithm of the odds of $z/(1 - z)$
```

```
plt.subplot(224)
```

```
plt.plot(x, y, 'b-',linewidth=3.0)
```

```
plt.yscale('logit')
```

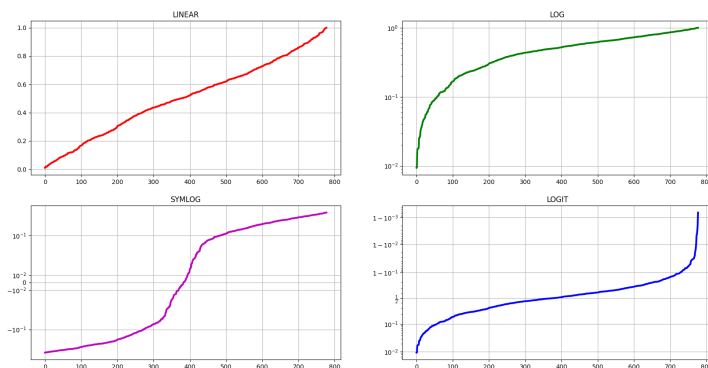
```
plt.title('LOGIT')
```

```
plt.grid(True)
```

```
Save and show
```

```
plt.savefig("Chart_13.png",transparent=True)
```

```
plt.show()
```



📍Something a bit more biological...

Let's look at the base composition in kilobase sliding windows along the first 100kb of the *E. coli* genome, stored as a

plain text file called `ecoli.txt`

```
Open the file, read it, and remove the newlines to get one long string
```

```
then take the first 100000 characters
```

```
ecoli = open("/localdisk/data/BPSM/Lecture19/ecoli.txt").read().replace('\n', '').upper()[0:100000]
```

```
Set sliding window size
```

```
window = 1000
```

```
Four lists to hold the numbers of each base
```

```
a = []
```

```
t = []
```

```
g = []
```

```
c = []
```

```
iterate over all the starting positions
```

```
for start in range(len(ecoli) - window):
```

```
get the current sliding window
```

```
 win = ecoli[start:start+window]
```

```
count each of the four bases and append to the list
```

```
 a.append(win.count('A') / window)
```

```
 t.append(win.count('T') / window)
```

```
 g.append(win.count('G') / window)
```

```
 c.append(win.count('C') / window)
```

```
Plot the lists with appropriate labels
```

```
plt.figure(figsize=(20,10))
```

```
plt.plot(a, label="A")
```

```
plt.plot(t, label="T")
```

```
plt.plot(g, label="G")
```

```
plt.plot(c, label="C")
```

```
plt.ylabel('Fraction of bases')
```

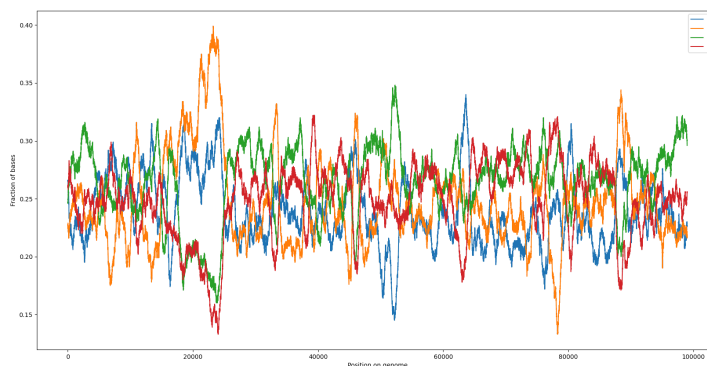
```
plt.xlabel('Position on genome')
```

```
plt.legend()
```

```
Save and show
```

```
plt.savefig("Chart_15.png",transparent=True)
```

```
plt.show()
```



By changing the code just a little bit, we can plot something else e.g. the fraction of repeated **di** nucleotides, one panel per base.

```
Open the file, read it, etc.
```

```
ecoli = open("/localdisk/data/BPSM/Lecture19/ecoli.txt").read().replace("\n", "").upper()[0:100000]
```

```
Set sliding window size
```

```
window = 1000
```

```
Four lists to hold the numbers of each dinucleotide
```

```
aa = []
```

```
tt = []
```

```
gg = []
```

```
cc = []
```

```
Iterate over all the starting positions
```

```
counter = 0
```

```
for start in range(len(ecoli) - window):
```

```
Get the current sliding window
```

```
 counter += 1
```

```
 print(counter)
```

```
 win = ecoli[start:start+window]
```

```
Count each of the four bases and append to the list
```

```
 aa.append(win.count('AA') / win.count('A'))
```

```
 tt.append(win.count('TT') / win.count('T'))
```

```
 gg.append(win.count('GG') / win.count('G'))
```

```
 cc.append(win.count('CC') / win.count('C'))
```

```
Plot the lists with appropriate labels
```

```
plt.figure(figsize=(20,10))
```

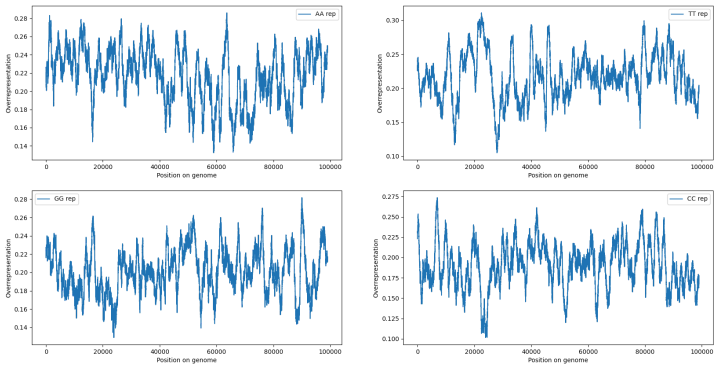
```
First panel
plt.subplot(221)
plt.plot(aa, label="AA rep")
plt.ylabel('Overrepresentation')
plt.xlabel('Position on genome')
plt.legend()

Second panel
plt.subplot(222)
plt.plot(tt, label="TT rep")
plt.ylabel('Overrepresentation')
plt.xlabel('Position on genome')
plt.legend()

Third panel
plt.subplot(223)
plt.plot(gg, label="GG rep")
plt.ylabel('Overrepresentation')
plt.xlabel('Position on genome')
plt.legend()

Fourth panel
plt.subplot(224)
plt.plot(cc, label="CC rep")
plt.ylabel('Overrepresentation')
plt.xlabel('Position on genome')
plt.legend()

Save and show
plt.savefig("Chart_16.png",transparent=True)
plt.show()
```



Or we can change the sliding window size, e.g. to 10000

```
Open the file
```

```
ecoli = open("ecoli.txt").read().replace("\n", '')[0:100000]
```

```
Set sliding window size
```

```
window = 10000
```

```
Four lists to hold the numbers of each nucleotide
```

```
a = []
```

```
t = []
```

```
g = []
```

```
c = []
```

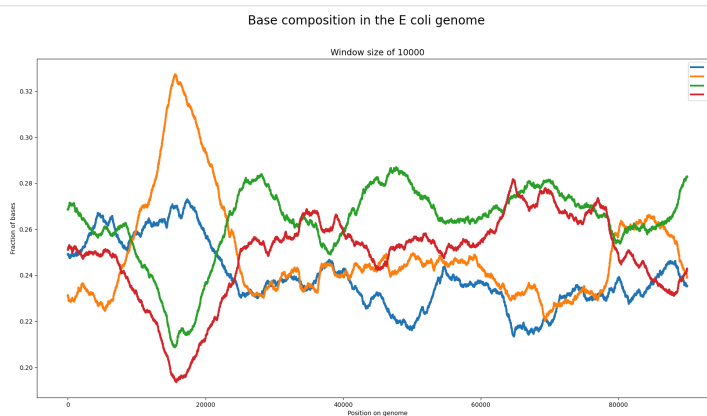
```
Iterate over all the starting positions
```

```
for start in range(len(ecoli) - window):
 win = ecoli[start:start+window]
 a.append(win.count('A') / window)
 t.append(win.count('T') / window)
 g.append(win.count('G') / window)
 c.append(win.count('C') / window)
```

```
Do the plotting bits
```

```
Python "responses" not shown
```

```
plt.figure(figsize=(20,10))
plt.plot(a, label="A",linewidth=3)
plt.plot(t, label="T",linewidth=3)
plt.plot(g, label="G",linewidth=3)
plt.plot(c, label="C",linewidth=3)
plt.ylabel('Fraction of bases')
plt.xlabel('Position on genome')
plt.suptitle("Base composition in the E coli genome",fontsize=20) # Not
plt.title("Window size of "+str(window),fontsize=14)
plt.legend()
plt.savefig("Chart_16A.png",transparent=True)
plt.show()
```



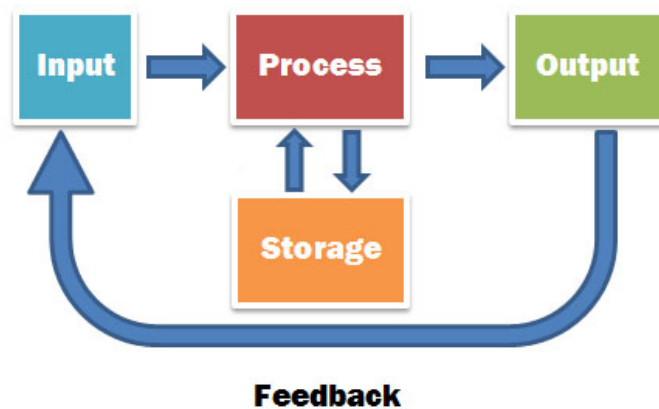
---

## 🏠 Exercises: your turn!

If you did not already do so in Python, make a directory within your Exercises space for today's work, and change directory to it.

```
mkdir ~/Exercises/Lecture19
```

```
cd ~/Exercises/Lecture19
```



The files required for today's exercise are on the server, in the directory

```
/localdisk/data/BPSM/Lecture19/
```

### 🏠 Exercise 1

- Use the `ecoli.txt` file and, using the code above as a starting point, make a chart which shows the AT content in a sliding 1000 base window
  - for the first 50000 bases
  - for the first 100000 bases
  - for the whole genome

### 🏠 Exercise 2

- Using the working code from the Exercise 1, write a script that calls a function that you have written to do the processing. The default sequence should be the `ecoli` one we used for the lecture. The user should be able to

provide the window size, whether they want AT content or GC content plotted, and what portion (base range) of the genome they want analysed

---

---

---

## POSSIBLE ANSWERS

---

- Use the `ecoli.txt` file and, using the code above as a starting point, make a chart which shows the AT content in a sliding 1000 base window
  - for the first 50000 bases
  - for the first 100000 bases
  - for the whole genome

**# Make a chart which shows the AT content in a sliding window**

# Written by s123456 on 19 Nov 2024

import os

import matplotlib.pyplot as plt

# Open the file, read it, and remove the newlines to get one long string

# then take the first 100000 characters

genome = open("/localdisk/data/BPSM/Lecture19/ecoli.txt").read().replace('\n', '').upper()

len(genome)

5498450

# Set sliding window size

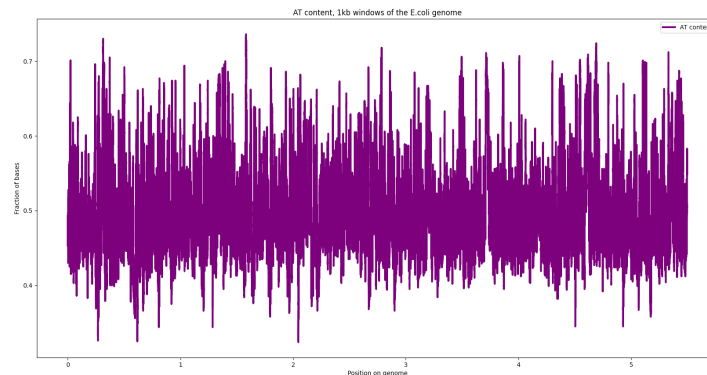
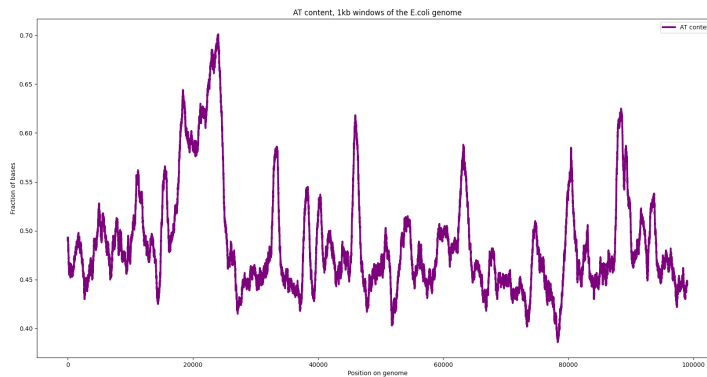
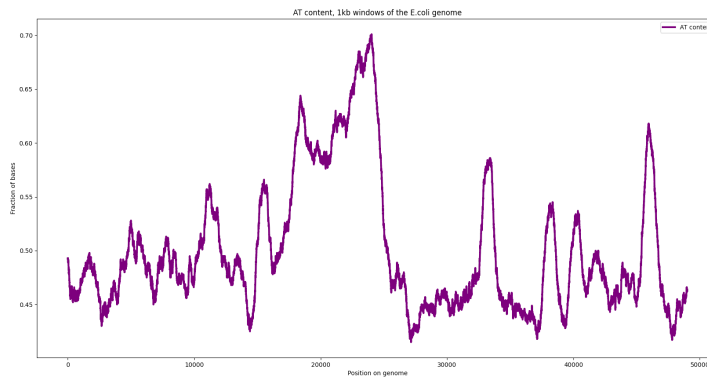
window = 1000

# Regions wanted

regions=[50000,100000,len(genome)]

# Iterate over all the regions

```
for region in regions :
 print("Processing the region from zero to",region)
 # A list to hold the numbers
 at = []
 # Iterate over all the starting positions
 for start in range(region - window):
 win = genome[start:start+window]
 at.append((win.count('A')+win.count('T')) / window)
 # Plot the list with appropriate labels
 plt.figure(figsize=(20,10))
 plt.plot(at, label="AT content",linewidth=3,color="purple")
 plt.ylabel('Fraction of bases')
 plt.xlabel('Position on genome')
 plt.title("AT content, 1kb windows of the E.coli genome")
 plt.legend()
 plt.savefig("Chart_Exercise_1_"+str(region)+".png",transparent=True)
 plt.show()
```



## Exercise 2

- Using the working code from the Exercise 1, write a script that calls a function that you have written to do the processing. The default sequence should be the ecoli one we used for the lecture. The user should be able to provide the window size, whether they want AT content or GC content plotted, and what portion (base range) of the genome they want analysed

What is the question actually asking you to do? *Your answers: ?*

What are the inputs? *Your answers: ?*

What processing is required? *Your answers: ?*

What are the outputs? *Your answers: ?*

What are the other "things" you should do? *Your answers: ?*

## # Local repository actions (the absolute minimum set of commands required here...)

### git st

On branch master

Changes not staged for commit:

```
(use "git add/rm <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
 deleted: ../Lecture04/CodeFiles.tar.gz
 modified: ../Lecture06/run_this_script_v1
```

Untracked files:

```
(use "git add <file>..." to include in what will be committed)
 ../BN.out
 ../BN.sh
 ../Lecture04/Als_ICA/
 ../Lecture04/all_the_things_I_did
 ../Lecture04/outs/
 ../Lecture05/Country.Mozambique.details
 ../Lecture07/Cosmoscarta.nuc.fa
 ../Lecture07/Cosmoscarta.nuc.gis
 ../Lecture07/myfileofaccessions.txt
 ../Lecture12/AJ223353.fasta
 ../Lecture12/AJ223353.genbank
 ../Lecture12/AJ223353_noheader.fasta
 ../Lecture12/AJ223353_noheader.fasta2
 ../Lecture12/AJ223353_noheader.fasta3
 ../Lecture12/All_exons.fasta
 ../Lecture12/All_noncodings.fasta
 ../Lecture12/local_exon01.fasta
 ../Lecture12/local_exon02.fasta
 ../Lecture12/local_intron01.fasta
 ../Lecture12/my_dna.txt
 ../Lecture12/plain_genomic_seq.txt
 ../Lecture12/quick_blast_check.out
 ../Lecture12/remote_exon01.fasta
 ../Lecture12/remote_noncoding01.fasta
 ../Lecture12/remote_noncoding02.fasta
 ../Lecture12/shutilversion.txt
 ../Lecture13/100_199/
 ../Lecture13/200_299/
 ../Lecture13/300_399/
 ../Lecture13/400_499/
 ../Lecture13/500_599/
 ../Lecture13/600_699/
 ../Lecture13/700_799/
 ../Lecture13/800_899/
 ../Lecture13/900_999/
 ../Lecture13/Cleaned_sequences.txt
 ../Lecture13/Cleaned_sequences2.txt
 ../Lecture13/Cleaned_sequences3.txt
 ../Lecture13/Exercise2_coding_sequence.fasta
 ../Lecture13/Lecture13.tar.gz
 ../Lecture13/dna_files/
 ../Lecture13/exons.txt
 ../Lecture13/genomic_dna2.txt
 ../Lecture13/input.txt
 ../Lecture13/remote_exon01.fasta
 ../Lecture14/__pycache__/
 ../Lecture14/data.csv
 ../Lecture16/__pycache__/
 ../Lecture17/All_the_Arabidopsis_ones.tsv
 ../Lecture17/eukaryotes.txt
 ../Lecture17/playtime/
 ./
```

no changes added to commit (use "git add" and/or "git commit -a")

ls

```
Chart_01.png Chart_05.png Chart_09.png Chart_13.png Chart_16.png
Chart_02.png Chart_06.png Chart_10.png Chart_14.png Chart_77.png
Chart_03.png Chart_07.png Chart_11.png Chart_15.png Lect19_ex1.py
Chart_04.png Chart_08.png Chart_12.png Chart_16A.png
```

# You might want to add other files?

**git add \*.py**

**git commit -m "Scripts from the python plots lecture"**

```
[master 75d3a16] Scripts from the python plots lecture
1 file changed, 36 insertions(+)
create mode 100644 Lecture19/Lect19_ex1.py
```

**git push AllLectures master**

```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 256 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 981 bytes | 327.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/B123456-2121/BPSM_LectureStuff.git
41e1ac8..75d3a16 master -> master
```

**git log**

```
75d3a16 (HEAD -> master, AllLectures/master) Scripts from the python plots lecture
41e1ac8 Scripts from the python regex lecture
097800e Script from the python pandas lecture
1d2319e Scripts from the python dictionaries lecture
c49df45 Scripts from the python functions lecture
3b8bf41 My scripts from the python conditionals exercises Py04
8f58f4c Scripts from the python lists and loops lecture 3
e444ef8 Scripts from the second BPSM python lecture
6d6f5e7 My python scripts from the first BPSM python lecture
eae65c6 (Lecture07/master) edirect seqs related stuff added
eea4f5e (Lec_6_branch) Kept bash script output files from Lecture 06
fba5cc0 Added run_this_script_v1 from BPSM Lecture06
ce2c15a nematode BLASTDB stuff added
19cdd2d Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeea I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

## Have your say...

The "Postbox of Truth": is your chance to give anonymous "in course" feedback/comments. Your comments are completely anonymous and confidential, so say what you think, irrespective of whether the comments are positive or otherwise! You can use this multiple times too: I use it throughout the course to enable you to give context-relevant feedback/make suggestions.

Just click the image!



[Sources used](#)